

MLOps & LLMOps

Milestone 2 Assignment

“Operationalizing Large Language Models”

Fall 2025

1. Goal

Milestone 2 extends your reproducible ML workflow into the world of **LLMOps** aka managing the lifecycle of Large Language Models. You will design, implement, and evaluate a complete *prompt engineering + RAG system* that demonstrates best practices in LLM operations: prompt experimentation, evaluation, guardrail, and monitoring.

By the end of this milestone, your system must:

- Compare multiple prompt strategies on a curated evaluation dataset.
- Implement a Retrieval-Augmented Generation (RAG) workflow with ingestion & inference pipelines.
- Integrate monitoring, evaluation, and safety guardrails into the LLM pipeline.
- Document, containerize, and automate everything through CI/CD.

2. Required Deliverables

All artefacts must be placed in the root of your public GitHub repository (tagged `v2.0-milestone2`) unless otherwise stated.

D1 Prompt Engineering Workflow

- Create an `experiments/prompts/` folder with **minimum of three** distinct prompting strategies for their chosen use case. These strategies must include:
 - (a) **Baseline:** A simple ‘Zero-Shot Prompting’ strategy.
 - (b) **Example-Driven:** ‘Few-Shot Prompting’. The analysis must test and report on the effect of the number of examples (e.g., $k = 3$ vs. $k = 5$).
 - (c) **Advanced Strategy:** At least *one* of the following:
 - ‘Chain-of-Thought (CoT)’ prompting (i.e., instructing the model to “think step-by-step”).
 - ‘Meta-Prompting’ (i.e., using a structured prompt that defines the model’s persona, rules, objectives, and output format).
- Compare them using a held-out evaluation dataset (include `data/eval.jsonl`).
- **Evaluation Metrics:** Prompts must be evaluated using *two* types of metrics:

- (a) **Quantitative:** An automated, objective metric appropriate for the task (e.g., 'ROUGE-L' for summarization, 'BLEU' for translation, or 'Embedding Cosine Similarity' against the ground truth for Q&A).
- (b) **Qualitative (Human-in-the-Loop):** A simple, student-defined rubric (e.g., a 1-5 scale for 'Factuality' and 'Helpfulness').
- (c) Log performance metrics (e.g., BLEU, ROUGE, factuality, relevance) using **Weights & Biases** or **MLflow**.
- (d) Produce a `prompt_report.md` summarizing:
 - Each strategy's structure + examples.
 - Quantitative + qualitative results.
 - Insights into prompt robustness and failure cases.

D2 RAG (Retrieval-Augmented Generation) Pipeline

- Implement a modular ingestion pipeline in `src/ingest.py` that indexes documents (e.g., PDFs, text) using **FAISS** / **Chroma** / **LlamaIndex**.
- Implement an inference API in `src/app.py` (FastAPI) where queries are answered via RAG retrieval + LLM response.
- Include diagrams:
 - **System Architecture Diagram** (draw.io or Mermaid) showing ingestion, retrieval, and generation flow.
 - **Data Flow Diagram** showing components interacting with cloud or local storage.
- Demonstrate reproducibility through a `Makefile` target (`make rag`) that runs the full pipeline end-to-end.

D3 Guardrails & Safety Mechanisms

- Implement content filters using **Guardrails AI**, **NeMo Guardrails**, or a custom policy engine.
- Define at least two rule types:
 - Input validation (e.g., PII detection, prompt injection filter).
 - Output moderation (e.g., toxicity threshold, hallucination filter).
- Log all guardrail events to your monitoring system.
- Document how guardrails integrate within your RAG pipeline.

D4 LLM Evaluation & Monitoring

- Track LLM metrics via **Prometheus** and visualize in **Grafana**.
- Monitor: `latency`, `token usage`, `cost`, and `guardrail violations`.
- Create an **Evidently Dashboard** or equivalent to monitor data drift in the retrieval corpus.
- Include screenshots / links to dashboards in **README.md**.

D5 CI/CD for LLMOps

- Extend your `.github/workflows/ci.yml` to include:
 - a. Linting + tests for prompt scripts.
 - b. Automated prompt evaluation on a small dataset.
 - c. Docker build & push for the RAG API.
 - d. Canary deployment of the LLM service.
- Achieve at least 80% coverage across unit + integration tests.

D6 Documentation & Reports

- Update **README.md** with:
 - Project overview & LLM Ops objectives.
 - Diagrams and links to dashboards.
 - Step-by-step RAG deployment guide.
 - API usage examples with sample queries.
- Add **EVALUATION.md** summarizing: evaluation methodology, prompt comparisons, and insights.

D7 Cloud Integration (Required)

- Use at least two cloud services (AWS, GCP, or Azure):
 - Example: AWS Lambda (for evaluation triggers), S3 (for indexed documents), EC2/Vertex AI (for model serving).
- Include configuration screenshots and setup steps in **README.md**.

D8 Security & Compliance

- Add **SECURITY.md** describing prompt injection defenses and data privacy.
- Run **pip-audit** for dependency scans (critical CVEs fail CI).
- Document how guardrails enforce responsible AI guidelines.

3. Submission Checklist

- Push to GitHub with tag **v2.0-milestone2**.
- Ensure CI/CD workflow passes.
- Submit public repo URL on LMS before **23:59 PKT on deadline**.

4. Bonus Paths (+5 pts)

Mention any of this if you have done in the README with relevant instructions

- Implement **LangChain** or **LlamaIndex** toolchains with custom retrievers.
- Add **A/B testing dashboard** comparing prompt variants.
- Deploy end-to-end system on a managed LLM platform (e.g., Vertex AI or Azure AI Studio).

5. Resources

- [LangChain Docs](#)
- [LlamaIndex Integrations](#)
- [Guardrails AI Docs](#)
- [MLflow](#)

- [Prometheus & Grafana](#)
- [Evidently AI](#)
- [GitHub Actions](#)